

Using a GX device as a remote toolbox

[Edit](#) [New page](#)[Jump to bottom](#)

mpvader edited this page 3 minutes ago · 11 revisions

1. Scope, purpose & warning

This document is intended for partners, such as Battery or BMS manufacturers, and explains how to use Victron Venus OS and GX products for remote diagnostics & firmware updates of your product.

Beware that the features explained in this document require quite capable development skills and will require a development investment from your company. You will have to develop, and provide us the sources of the required commandline tools which need to be added to Venus OS. From Victron side, we only provide support for questions.

2. Introduction

Imagine a support engineer logging on to the shell of our device, remotely over the internet, and then use commandline tools to interface to your product either via canbus, a serial connection such as RS485 or RS232; or any other port. To download detailed log files, change factory parameters or do a firmware update. All without travelling and/or explaining an installer over the phone how to perform the procedure with laptop and special cables.

Also, when the extra required effort is worth it; we can discuss integrating the these features into the VRM Portal; so that it can be used without requiring your remote support engineer to login. Normal users and installers can do it themselves; just like they can today update their Victron Solar chargers, for example.

For Victron, such remote power has proven of extreme value. And it all revolves around the GX products, Venus OS and the VRM Portal. An introduction of those terms:

- The [GX Product range](#) consists of multiple variants of devices; all having ethernet and/or WiFi to connect to the internet, USB ports, CAN-Bus ports, as well as various other ports specific to Victron products.
- Venus OS is the software that runs on each of them. To learn more about Venus OS, start reading [this public wiki page](#).
- The [VRM Portal](#) is our online remote monitoring portal.

See our [Remote VEConfigure manual](#) and our [Remote Firmware Update manual](#) for examples on how we offer these features for Victron products.

In case you do a lot of work on Linux; and already have commandline tools available; then it might be as easy as just a [one-time cross compile](#) to get the basics working and adding value.

3. Remote Support access

The Remote Support feature allows accessing the Linux Shell remotely, over the internet.

And it works without having to change firewall settings in the installation, or setup (permanent) VPNs: that is already available and provisioned. including those installed in the field, behind firewalls. By default, this feature is disabled. The user needs to enable it in the menus: Settings → General → Remote support (SSH)'. This feature consists of a permanent reverse-ssh tunnel between the Venus device and the Support-host (supporthost.victronenergy.com), which works as a pivot-server.

Besides logging in, and doing all that can be done via the command-line, it is (ofcourse) also possible to upload and download files and tools.

For more information about the the Remote Support feature, ask us for the *Venus OS - Remote Support Shell Access* documentation. First though, discover the possibilities by logging into a device local to you network and experiment. Instructions for doing so are in our [Venus OS Root Access document](#).

Note that to be able to login remotely; you will need to have provisioned the device with your ssh key; or have Remote Console access, and temporarily set a password for the root account.

4. Firmware update from VRM framework

4.1 Introduction

To understand this, start with going through the process as a user will see it: get yourself access to a Victron system that is on VRM; then go to Device list, and press the firmware update button on the bottom. [Remote firmware update user manual](#).

Process explained in short: first VRM issues the list command to the GX device. Locally on the GX device, a python script runs multiple command line executables; that discover all devices connected to it. On purpose it does not use the normal device drivers for this; since they might not find devices in bootloader mode; or might not find each individual BMS in case of a multiple-BMS system.

The list of discovered devices is sent back to the web-browser; where the user can then take action & initiate a firmware update on one of the devices.

When updating, the file is transferred to the GX device first, and only thereafter it will initiate the update.

Firmware library: at Victron we maintain a firmware library. That is what the VRM portal uses to suggest the user that there is a new firmware available.

Product id: in the XML; a product id is used. Ask Victron for your productid(s).

4.2 How this works in detail

A request to list the devices is received by mqtt-rpc.py. Login to a GX device to see its sources. Make sure to use the most recent firmware version when doing so. It calls various commandline executables, or scripts, to scan the ports for connected update-able devices.

Those tools each output a certain XML format. Their return code indicates success (0) or failure (<> 0).

As part of the list; two parameters are sent to the browser; and then sent back when initiating an update, which purpose it is to identify the device:

1. connection-type: currently there are two types: direct & can.
2. connection-id: for a canbus connection that would be the address on the bus. For a serial connected device it would be the tty port. Note that this is free format: all is fine, as long as the executable that performs the firmware update understands it. For example; if ever its necessary to support updating modbusrtu connected devices; that could be done by using a combination of the tty and the modbus-id, for example `"/dev/ttyUSB12%%12"`.

Requirements:

- commandline executable; or script.
- It will be called from mqtt-rpc.py; login to a GX device to see the source of that (use the most recent firmware version!)
- Output is XML & a return code signalling OK (0) or an error (<> 0). See below for details on XML output
- Make sure to disable buffered output; use line buffering.
- When using canbus: be aware that most devices have multiple canbusses: there needs to be a -c commandline parameter that lets the python script select which canbus interface the executable should use. It does so based on the profiles selected by the installer in the Settings -> Services -> Canbus profiles menu.
- When using canbus: don't change baudrate. The installer has already selected a baudrate in the Canbus profiles menu.
- Two types of calls are made
 - i. List: needs to listen on the supplied canbus or serial ports. And output the found BMS-es.
 - ii. Update: update firmware. See the `respond_to_deviceupdate()` function in mqtt-rpc.py.

Active vs passive probing

(this section needs to find a better place or be reworded)

To find your device; its not allowed to do what we call active probing: don't send any data to a tty; unless already the system can be sure that that tty has your product connected.

For serial ports; this means that there are only two ways: 1) based on USB ID_MODEL. 2) by listening. This can't take too long though (few seconds max).

We strongly prefer USB ID_MODEL. For RS232 this can be done by mandating the use of a certain usb-serial cable; and reprogramming its ID before stocking/installing. This is also the technique we use for VE.Direct USB cables, and MK3-USB devices. See the references to ID_MODEL and `ft_prog` for `ftdi` in this document for more info.

For canbus: it means you can request data by broadcast; but not send any data to an unknown device.

List command examples

```
# In case its tty connected:
acme-tool -s "/dev/ttyS1 /dev/ttyUSB12"

# in case its serial port connected:
acme-tool -c "can1"
```

Note how the serial port scan command supports scanning multiple ports. Doing so is not mandatory; the alternative is that its called several times, once for each tty, from the Python script. For the canbus variant, note how the tool supports selecting a canbus interface. This is required; since most GX devices have multiple canbus interfaces.

To see an example of the XML output, login to a GX product and run `vup --xml` .

Update command examples

```
acme-tool -s "/dev/ttyS1" -f "/data/cache/myfile.bin"
acme-tool -c can1 -f "/data/cache/myfile.bin" -n 0x24
```

For more details, login to a GX product and run `vup --help` .

+ Add a custom footer

▼ Pages 114
Find a page...
▶ Home
▶ CAN Bus testing
▶ CANvu GX
▶ CANvu GX IO Extender hw changelog
▶ CCGX hw changelog
▶ Cerbo GX cable
▶ Cerbo GX hw changelog
▶ Cerbo GX info
▶ Cerbo GX testing

▸ cerbo meeting notes 20190404
▸ cerbo meeting notes 20190508
▸ cerbo meeting notes 20190909
▸ cerbo meeting notes 20191106
▸ cerbo meeting notes 20200228
▸ Cerbo S GX hw changelog
Show 99 more pages...

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/victronenergy/venus-private/wiki.git>